

UI 1 - Incident Dashboard and Assign Responder

INC-2841 — Production API timeout

Critical

Opened 14 min ago · SLA: 46 min remaining

Affected service

Payments API

Incident Commander

JL Jamie L.

CURRENT RESPONDERS

SR Sam R.

Backend Eng · on-call

Active

No second responder assigned

ASSIGN RESPONDER

On-call group

Payments team

Responder

— select —

Note (optional)

e.g. needs DB access for investigation

Assign responder

Cancel

UI 2 - Reassign Responder

Reassign responder

INC-2841 · currently assigned to Sam R.

Sam R. will be removed. Ensure continuity before reassigning.

Current responder

SR Sam R.

Backend Eng

Reassign to

— select new responder —

Reason for reassignment

e.g. Sam is unavailable — end-of-shift handoff

Confirm reassignment

Cancel

UI 3 - Remove Responder

Remove responder

INC-2841 · this action will be logged

SR

Sam R.
Backend Eng · assigned 22 min ago

Reason for removal *

— select reason —

Resolved / Wrong group paged / Unavailable / Other

Additional notes

Optional details

Cannot remove the only responder.
Assign a replacement before removing.

Remove responder

Cancel

↓

UI 4 - View Responders and Assignment History

INC-2841 — Production API timeout

Critical

Read-only view · stakeholder

ACTIVE RESPONDERS

JL

Jamie L.
Incident Commander · Payments team

Commander

AT

Alex T.
Backend Eng · reassigned 8 min ago

Active

MD

Morgan D.
SRE · assigned 14 min ago

Active

ASSIGNMENT HISTORY

●

Reassigned: Sam R. → Alex T.
By Jamie L. · Reason: end-of-shift handoff

8 min ago

●

Assigned: Morgan D.
By Jamie L. · SRE on-call group

14 min ago

●

Assigned: Sam R.
By system · Payments team auto-page

22 min ago

●

Incident created · severity: Critical
Commander assigned: Jamie L.

36 min ago

Algorithm 1: The assignResponder() use case

Input: incidentId, responderId, groupId, actorId (the incident commander performing the action)

Output: Updated incident record, logged AssignmentEvent, sent Notification

```
1.1 incident ← load incident record for incidentId
    /* 1. validate incident state */
1.2 if incident is None then
1.3     return error: incident not found
1.4 if incident.status = Resolved or incident.status = Closed then
1.5     return error: cannot assign to a resolved or closed incident
    /* 2. validate the responder */
1.6 responder ← load responder record for responderId
1.7 if responder is None then
1.8     return error: responder not found
1.9 if responderId ∈ incident.assignedResponderIds then
1.10    return error: responder is already assigned to this incident
    /* 3. perform assignment */
1.11 add responderId to incident.assignedResponderIds
    /* 4. log the event */
1.12 event ← new AssignmentEvent(incidentId, eventType = Assigned,
    actorId, affectedResponderId = responderId, timestamp = now())
1.13 persist event
    /* 5. notify relevant parties */
1.14 recipients ← {responderId} ∪ getNotifyList(incident)
1.15 foreach r in recipients do
1.16     sendNotification(r, incidentId, message = "Responder assigned")
1.17 persist updated incident
1.18 return success
```

Algorithm 2: The reassignResponder() use case

Input: incidentId, currentResponderId, newResponderId, reason, actorId

Output: Updated incident record, logged AssignmentEvent, sent Notification

```
2.1 incident ← load incident record for incidentId
    /* 1. validate inputs */
2.2 if incident is None or incident.status = Closed then
2.3     return error: invalid or closed incident
2.4 if currentResponderId ∉ incident.assignedResponderIds then
2.5     return error: current responder is not assigned to this incident
2.6 newResponder ← load responder record for newResponderId
2.7 if newResponder is None then
2.8     return error: replacement responder not found
2.9 if newResponderId ∈ incident.assignedResponderIds then
2.10    return error: replacement is already assigned to this incident
```

```

        /* 2. perform reassignment */
2.11 remove currentResponderId from incident.assignedResponderIds
2.12 add newResponderId to incident.assignedResponderIds
        /* 3. log the event */
2.13 event ← new AssignmentEvent(incidentId, eventType = Reassigned,
        actorId, affectedResponderId = currentResponderId,
        replacedById = newResponderId, reason = reason, timestamp = now())
2.14 persist event
        /* 4. notify both parties */
2.15 sendNotification(currentResponderId, incidentId, message = "You have been
removed")
2.16 sendNotification(newResponderId, incidentId, message = "You have been
assigned")
2.17 persist updated incident
2.18 return success

```

Algorithm 3: The removeResponder() use case

Input: incidentId, responderId, reason (reason code), actorId

Output: Updated incident record, logged AssignmentEvent, sent Notification

```

3.1 incident ← load incident record for incidentId
        /* 1. validate incident and responder */
3.2 if incident is None or incident.status = Closed then
3.3     return error: invalid or closed incident
3.4 if responderId ∉ incident.assignedResponderIds then
3.5     return error: responder is not assigned to this incident
        /* 2. check that a valid reason is provided */
3.6 if reason is empty or reason ∉ VALID_REASON_CODES then
3.7     return error: a valid removal reason is required
        /* 3. guard against leaving the incident unattended */
3.8 if |incident.assignedResponderIds| = 1 then
3.9     return error: cannot remove the only responder; assign a replacement
first
        /* 4. perform removal */
3.10 remove responderId from incident.assignedResponderIds
        /* 5. log the event */
3.11 event ← new AssignmentEvent(incidentId, eventType = Removed,
        actorId, affectedResponderId = responderId,
        reason = reason, timestamp = now())
3.12 persist event
        /* 6. notify the removed responder */
3.13 sendNotification(responderId, incidentId, message = "You have been
removed")
3.14 persist updated incident
3.15 return success

```

Algorithm 4: The viewIncidentResponders() use case

Input: incidentId, requestorId (ID of the user requesting the view)

Output: Incident summary, list of active responders, ordered list of AssignmentEvents

```
4.1  incident ← load incident record for incidentId
    /* 1. validate access */
4.2  if incident is None then
4.3      return error: incident not found
4.4  requestor ← load user record for requestorId
4.5  if not hasViewPermission(requestor, incident) then
4.6      return error: access denied
    /* 2. build the responder list */
4.7  responderList ← empty list
4.8  foreach id in incident.assignedResponderIds do
4.9      r ← load responder record for id
        /* 3. fill each entry with name, role, group, and assignment time */
4.10  entry ← {name: r.name, role: r.role, group: r.onCallGroupName,
            assignedAt: getAssignedTimestamp(id, incidentId)}
4.11  append entry to responderList
    /* 4. load assignment history, most recent first */
4.12  eventLog ← query AssignmentEvents where incidentId = incidentId,
            ordered by timestamp descending
    /* 5. return the complete incident view */
4.13  return {incident: incident, responders: responderList, history: eventLog}
```